Day 169 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 2 reference

Watch: Gaussian Processes and Bayesian Optimization from Physicists' Perspective (Machine Learning in the Nanoworld) | Bayesian Optimization: expected improvement (+LCB, PI, EV) interpretation (A. Sala)

`day169_bayesian_optimisation.py`, segment by segment, in the order `main()` uses things. Today's new parts are normal_cdf, normal_pdf, matern52, gp_fit, gp_predict, fit_gp, expected_improvement and propose, copied unchanged into day169_exercise.py; Day 168's parts and its evaluate() are imported. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = trials (rows in the unit cube) · y = their scores · z = standardised scores · K = the trials' kernel matrix, L its Cholesky factor · ell = length scale · mu, sd = the GP's mean and standard deviation · C = candidates · s = the pool's tuning scores

Segment 0 — Setup

Imports and constants

CODE

```
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(ROOT, "week-16-classical-ml-trees"))
sys.path.insert(0, os.path.join(ROOT, "week-24-model-evaluation-statistics"))
with contextlib.redirect_stdout(io.StringIO()):
    import day168_exercise as d168
    import day168_grid_vs_random_search as L168
    import day161_exercise as d161
ELLS = (0.05, 0.1, 0.2, 0.3, 0.5, 0.8, 1.2, 2.0)
NOISE = 1e-4
N0 = 5
POOL = 200
BUDGET = 64
REPEATS = 100
CONT_RUNS = 3
CONT_BUDGET = 30
HMIN = -3.86278
```

- `sys.path.insert(0, HERE)` — this week's own folder first, so Day 168's files import by name.
- `import day168_grid_vs_random_search as L168` — a lesson can be imported too: main() runs only under __name__ == "__main__".
- `ELLS = (0.05, ..., 2.0)` — the length scales tried; the unit cube is 1 wide, so 0.05 is very wiggly and 2.0 nearly flat.

Segment 1 — The parts

The normal distribution without SciPy

MATH

$$\Phi(z) = \tfrac{1}{2}(1 + erf(z/\sqrt{2})) \quad \phi(z) = e^{-z^2/2} / \sqrt{(2\pi)}$$

CODE

```
_erf = np.vectorize(math.erf)
def normal_cdf(z):
    return 0.5 * (1 + _erf(np.asarray(z, float) / np.sqrt(2)))
def normal_pdf(z):
    return np.exp(-0.5 * np.asarray(z, float) ** 2) / np.sqrt(2 * np.pi)
```

- `np.vectorize(math.erf)` — wraps a one-number function so it accepts arrays (a convenience loop, not faster).
- `_erf` — a leading underscore marks a name as internal to the module.
- `np.asarray(z, float)` — accepts a number, a list or an array.

matern52 and gp_fit

MATH

$$K = LL^T, \quad a = K^{-1}z \quad log\ p(z) = -\tfrac{1}{2}\, z^T a - \Sigma\ log\ L_{ii} - (n/2)\ log\ 2\pi$$

CODE

```
def matern52(A, B, ell):
    d = np.sqrt(np.maximum(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1), 0)) / ell
    return (1 + np.sqrt(5) * d + 5 / 3 * d ** 2) * np.exp(-np.sqrt(5) * d)
def gp_fit(X, y, ell, noise=NOISE):
    m = y.mean(); sd = y.std() if y.std() > 0 else 1.0
    z = (y - m) / sd
    K = matern52(X, X, ell) + noise * np.eye(len(X))
    L = np.linalg.cholesky(K)
    a = np.linalg.solve(L.T, np.linalg.solve(L, z))
    lml = -0.5 * z @ a - np.log(np.diag(L)).sum() - 0.5 * len(X) * np.log(2 * np.pi)
    return dict(X=X, L=L, a=a, m=m, sd=sd, ell=ell, lml=lml)
```

- `A[:, None, :] - B[None, :, :]` — broadcasting to rows of A × rows of B × settings: every pair's difference at once.
- `np.maximum(..., 0)` — guards the square root against tiny negative rounding.
- `y.std() if y.std() > 0 else 1.0` — avoids dividing by zero when all scores are equal.
- `np.linalg.cholesky(K)` — K = L L^T with L lower-triangular; fails loudly if K is not positive definite, which the noise term prevents.
- `np.linalg.solve(L.T, np.linalg.solve(L, z))` — $K^{-1}z$ in two triangular steps, without forming the inverse.
- `np.log(np.diag(L)).sum()` — half of log|K|: the determinant of a triangular matrix is the product of its diagonal.

gp_predict and fit_gp

MATH

$$mean = k^T K^{-1} z \quad var = 1 - k^T K^{-1} k$$

CODE

```
def gp_predict(g, Xs):
    Ks = matern52(Xs, g["X"], g["ell"])
    mu = Ks @ g["a"]
    v = np.linalg.solve(g["L"], Ks.T)
    var = np.maximum(1 - (v * v).sum(0), 1e-12)
    return g["m"] + g["sd"] * mu, g["sd"] * np.sqrt(var)
def fit_gp(X, y, ells=ELLS):
    return max((gp_fit(X, y, e) for e in ells), key=lambda g: g["lml"])
```

- `Ks @ g["a"]` — the posterior mean: each new point's similarities times $K^{-1}z$.
- `v = np.linalg.solve(g["L"], Ks.T)` — $L^{-1}k$ for every new point; (v * v).sum(0) is $k^T K^{-1} k$.
- `g["m"] + g["sd"] * mu` — undoes the standardisation.
- `max(..., key=lambda g: g["lml"])` — a generator of fitted GPs; key picks the one with the highest log marginal likelihood.

expected_improvement and propose

MATH

$$EI = (best - \mu)\Phi(z) + \sigma\phi(z), \quad z = (best - \mu)/\sigma$$

CODE

```
def expected_improvement(mu, sd, best):
    z = (best - mu) / sd
    return (best - mu) * normal_cdf(z) + sd * normal_pdf(z)
def propose(X, y, C):
    g = fit_gp(X, y)
    mu, sd = gp_predict(g, C)
    return int(np.argmax(expected_improvement(mu, sd, y.min()))), g
```

- `(best - mu) * normal_cdf(z) + sd * normal_pdf(z)` — two terms: a low mean (exploitation) and a large sd (exploration).
- `return int(...), g` — returning a tuple: the chosen index and the GP, so callers can inspect it.

Test functions and candidates

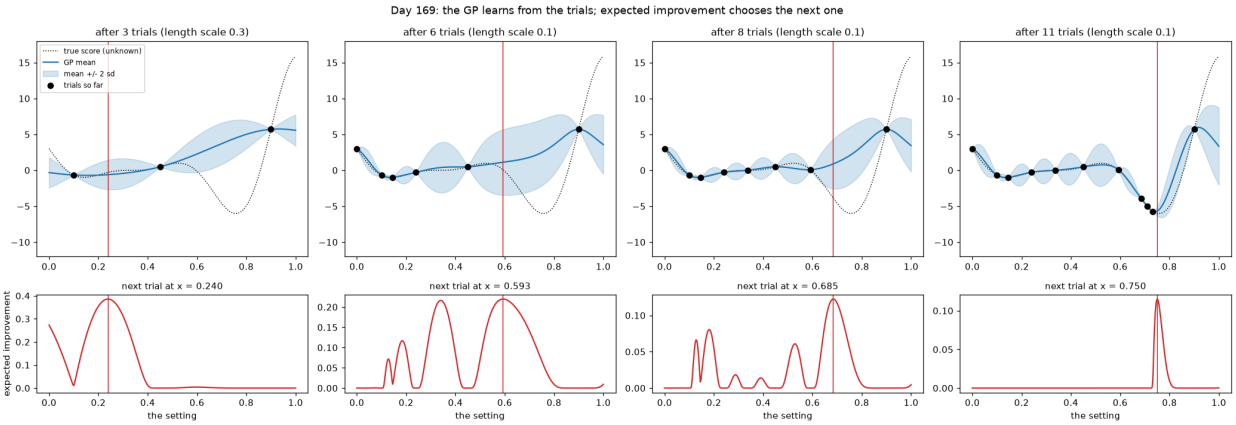
CODE

```
def forrester(x):
    return (6 * x - 2) ** 2 * np.sin(12 * x - 4)
H_ALPHA = np.array([1.0, 1.2, 3.0, 3.2])
H_A = np.array([[3, 10, 30], [0.1, 10, 35], [3, 10, 30], [0.1, 10, 35.]])
H_P = 1e-4 * np.array([[3689, 1170, 2673], [4699, 4387, 7470], [1091, 8732, 5547], [381, 5743,
    8828.]])
def hartmann3(X):
    return -np.sum(H_ALPHA * np.exp(-np.sum(H_A * (X[:, None, :] - H_P) ** 2, -1)), -1)
def candidates(X, y, rng, n=2000, near=500):
    return np.vstack([rng.rand(n, X.shape[1]), np.clip(X[np.argmin(y)] + 0.05 * rng.randn(near,
        X.shape[1]), 0, 1)])
```

- `np.sum(H_A * (X[:, None, :] - H_P) ** 2, -1)` — points × 4 dips × 3 settings, summed over settings: each point's weighted distance to each dip.

- `np.clip(X[np.argmin(y)] + 0.05 * rng.randn(near, ...), 0, 1)` — 500 candidates near the best trial, kept inside the cube.
- `np.vstack([...])` — random candidates and nearby ones stacked into one array.

SYSTEM DIAGRAM



Bayesian optimisation on the 1-D test function, four moments.

Segment 2 — The experiments

demo_gp: Bayesian optimisation in 1-D

CODE

```
X = np.array([[0.1], [0.45], [0.9]]); y = forrester(X[:, 0])
for _ in range(12):
    i, g = propose(X, y, xs); X = np.vstack([X, xs[i]]); y = np.r_[y, forrester(xs[i:i + 1, 0])]
print("  Bayesian optimisation from trials at 0.1, 0.45 and 0.9; trials 4 to 15 at " + ",
    ".join(f"{v:.3f}" for v in X[3:, 0]))
print("  best score after each of trials 4 to 15: " + ", ".join(f"{v:.2f}" for v in
    d168.best_so_far(y)[3:]))
```

- `for _ in range(12):` — the underscore names a loop variable that is not used.
- `np.vstack([X, xs[i]]); np.r_[y, ...]` — add one row to the trials and one value to the scores.
- `", ".join(f"{v:.3f}" for v in X[3:, 0])` — a generator inside join: every trial after the first three, formatted.

demo_ei: checking a formula by simulation

CODE

```
rng = np.random.RandomState(0)
for mu, sd, best in ((0.0, 1.0, 0.0), (1.0, 0.5, 0.0), (-0.5, 0.2, 0.0), (0.3, 2.0, 0.0)):
    draws = mu + sd * rng.randn(1_000_000)
    mc = np.mean(np.maximum(best - draws, 0))
```

- `1_000_000` — underscores in a number literal are ignored; they only make it readable.
- `np.maximum(best - draws, 0)` — the improvement in each simulated world, 0 when the draw is no better.

bo_pool: the loop (bo_continuous is the same loop over fresh candidates)

CODE

```
def bo_pool(U, s, n, seed, acq="ei"):
    rng = np.random.RandomState(seed); idx = list(rng.choice(len(U), N0, replace=False))
    while len(idx) < n:
        rest = np.setdiff1d(np.arange(len(U)), idx)
        if acq == "ei":
            i, _ = propose(U[idx], s[idx], U[rest])
        else:
            mu, sd = gp_predict(fit_gp(U[idx], s[idx]), U[rest])
            i = int(np.argmin(mu)) if acq == "mean" else int(np.argmax(sd))
        idx.append(int(rest[i]))
    return d168.best_so_far(s[idx])
```

- `rng.choice(len(U), N0, replace=False)` — five distinct random configurations to start.
- `np.setdiff1d(np.arange(len(U)), idx)` — the configurations not yet tried.
- `int(np.argmin(mu)) if acq == "mean" else int(np.argmax(sd))` — the greedy and explore-only rules, as one conditional expression.
- `idx.append(int(rest[i]))` — i indexes the untried list; rest[i] turns it back into a pool index.

demo_pool: the shuffled control and what the GP knows

CODE

```
sh = np.random.RandomState(9)
C["shuffled"] = np.array([bo_pool(U, s[sh.permutation(POOL)], BUDGET, k) for k in range(REPEATS)])
reach = lambda c: int(np.argmax(c <= s.min() + 0.001)) + 1 if np.any(c <= s.min() + 0.001) else
    BUDGET + 1
first, firstr = [reach(c) for c in C["ei"]], [reach(c) for c in C["random"]]
```

- `s[sh.permutation(POOL)]` — the same scores in a random order: every configuration gets another's score.
- `reach = lambda c: ... if np.any(...) else BUDGET + 1` — a conditional inside a lambda: the first trial within 0.001, or 65 if never.

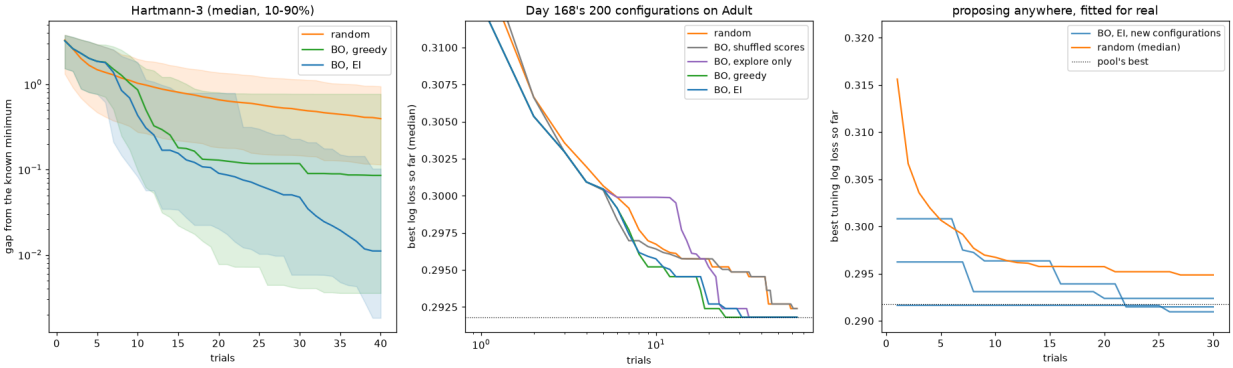
demo_real: proposing anywhere, and repeats

CODE

```
c = d168.decode(Cn[i:i + 1])[0]; key = tuple(c.values())
if key in seen:
    r = seen[key]; dup += 1
else:
    r = L168.evaluate([c], X, y, A, V, F)[0]; seen[key] = r
```

- `Cn[i:i + 1]` — a slice keeps the 2-D shape decode expects (one row).
- `tuple(c.values())` — a dictionary's values as a tuple: hashable, so it can be a key in seen.
- `if key in seen:` — a dictionary lookup: has this exact configuration been fitted already?

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.vectorize(f)` — makes a one-value function accept arrays.
- `np.linalg.cholesky(K)` — the lower-triangular L with K = LL^T.
- `max(iterable, key=f)` — the item with the largest f(item).
- `np.setdiff1d(a, b)` — items of a not in b, sorted.
- `1_000_000` — underscores as digit separators.
- `tuple(d.values())` — a hashable key built from a dictionary.

End of Day 169 syntax reference. For the experiments and what they showed, see the Day 169 complete guide.